



Reachability Set Generation Using Hybrid Relation Compatible Saturation

Shruti Biswal^(✉) and Andrew S. Miner

Iowa State University, Ames, IA 50011, USA
{sbiswal,asminer}@iastate.edu

Abstract. Generating the state space of any finite discrete-state system using symbolic algorithms like saturation requires the use of decision diagrams or compatible structures for encoding its reachability set and transition relations. For systems that can be formally expressed using ordinary Petri Nets (PN), implicit relations, a static alternative to decision diagram-based representation of transition relations, can significantly improve the performance of saturation. However, in practice, some systems require more general models, such as self-modifying Petri nets, which cannot currently utilize implicit relations and thus use decision diagrams that are repeatedly rebuilt to accommodate the changing bounds of the system variables, potentially leading to overhead in saturation algorithm. This work introduces a hybrid representation for transition relations, that combines decision diagrams and implicit relations, to reduce the rebuilding overheads of the saturation algorithm for a general class of models. Experiments on several benchmark models across different tools demonstrate the efficiency of this representation.

Keywords: Petri Nets · Self-modifying nets · Decision diagrams · Saturation · Reachability set generation · Implicit relations

1 Introduction

With rapid progress in the development of automated systems, formal verification techniques have increasingly aided in building thorough and reliable automation. One such technique, model checking, involves an exhaustive analysis of all possible systemic behavior, conventionally defined in terms of its variables and events, for a design to be verified. This motivates the need for efficient state-space generation methods, in spite of the fact that the reachability set of a system can be extremely large due to state explosion problem.

State-of-the-art symbolic techniques for state-space generation, such as *saturation* [4], are time- and memory-efficient with respect to the explicit methods. Several implementations of the saturation algorithm demonstrate that the technique has been often fine-tuned to perform efficiently with different sub-classes of discrete-state systems. The most generic implementation of the saturation algorithm utilizes multi-valued decision-diagrams (MDDs) [13] as the underlying data structure for representation of reachable states and extensible matrix

diagrams (MxDs) [4, 20] for representation of system events where the bounds of the states variables are not known a priori. For systems that can be represented in terms of Kronecker products, a static encoding for the events, such as *implicit relation forests* [3], is an efficient alternative that aids saturation by eliminating the need to rebuild transition relations as state variable sizes increase, thereby allowing the algorithm to focus completely on generation of reachability set.

In the real world, however, systems may have some events that are not expressible as Kronecker products, or are expressible as Kronecker products except for a few components. Such models would require either to use matrix diagrams entirely, or to split events until each event is expressible as a Kronecker product so that implicit relation forests may be used. The aim of the paper is to explore a *hybrid relation*, that allows the mixture of implicit nodes (from implicit relation forests) and matrix diagram nodes in the same forest. This makes it possible to simultaneously use implicit nodes for (portions of) events expressible as a Kronecker product, and matrix diagram nodes for the more general case.

The remainder of the paper is organized as follows. Section 2 describes the formalism for the class of models befitting the proposed methodology, the saturation algorithm and briefly discusses the various saturation-compatible data structures available in the literature. Section 3 introduces hybrid relation forests, describes their construction from a model, and formulates a modified saturation algorithm that works with events represented in a hybrid relation forest. Section 4 evaluates the proposed method experimentally by comparing its performance with existing techniques in the literature. Finally, Sect. 5 concludes the paper and presents some directions for future work.

2 Background and Related Work

In order to describe the problem domain, we use a standard definition of extended Petri nets, which are quite general and include inhibitor arcs and marking-dependent arc cardinalities, as a high-level formalism to define the class of finite discrete-state models.

Definition 1 (Extended Petri Net). The high-level formalism of a model $\mathcal{M}$ as an extended Petri net is defined as a tuple $(\mathcal{P}, \mathcal{T}, \mathcal{F}^-, \mathcal{F}^+, \mathcal{F}^o, \mathbf{i}_0)$ where:

- $\mathcal{P} = \{p_1, p_2, \dots, p_{|\mathcal{P}|}\}$ is a finite set of *places* that encode the state variables of $\mathcal{M}$. Each place can contain a natural number of tokens. A marking $\mathbf{i} = (i_1, i_2, \dots, i_{|\mathcal{P}|}) \in \mathbb{N}^{|\mathcal{P}|}$ represents the number of tokens in each place.
- $\mathcal{T} = \{t_1, t_2, \dots, t_{|\mathcal{T}|}\}$ is a finite set of *transitions* of the net that represent the events of $\mathcal{M}$.
- $\mathcal{F}^- : \mathcal{P} \times \mathcal{T} \times \mathbb{N}^{|\mathcal{P}|} \rightarrow \mathbb{N}$, $\mathcal{F}^+ : \mathcal{T} \times \mathcal{P} \times \mathbb{N}^{|\mathcal{P}|} \rightarrow \mathbb{N}$, and $\mathcal{F}^o : \mathcal{P} \times \mathcal{T} \times \mathbb{N}^{|\mathcal{P}|} \rightarrow \mathbb{N} \cup \{\infty\}$ are the marking-dependent input, output, and inhibitor arcs, respectively.
- $\mathbf{i}_0 \in \mathbb{N}^{|\mathcal{P}|}$ is the initial marking of the net.

A transition t is *enabled* on a marking $\mathbf{i}$ iff $\forall p \in \mathcal{P}, \mathcal{F}^-(p, t, \mathbf{i}) \leq i_p < \mathcal{F}^o(p, t, \mathbf{i})$. When multiple transitions are enabled on a marking, the choice for *firing* a transition is made non-deterministically. On firing an enabled transition t on $\mathbf{i}$, a new marking $\mathbf{j}$ is obtained, where $\mathbf{j}_p = \mathbf{i}_p - \mathcal{F}^-(p, t, \mathbf{i}) + \mathcal{F}^+(p, t, \mathbf{i}), \forall p \in \mathcal{P}$.

The next-state function $\mathcal{N}_t(\mathcal{I}) = \{\mathbf{j} : \mathbf{i} \in \mathcal{I} \text{ and } \mathbf{i} \xrightarrow{t} \mathbf{j}\}$ represents the next set of markings obtained when transition t is fired on a given set of markings, $\mathcal{I}$. The support-set of a next-state function, defined as $\text{supp}(\mathcal{N}_t) = \{p \in \mathcal{P} : \exists \mathbf{i} \in \mathbb{N}^{|\mathcal{P}|}, \mathcal{F}^-(p, t, \mathbf{i}) > 0 \vee \mathcal{F}^+(p, t, \mathbf{i}) > 0 \vee \mathcal{F}^o(p, t, \mathbf{i}) < \infty\}$, is the set of places that could affect the enabling of t , or could affect or be affected by the firing of t .

Given the initial marking $\mathbf{i}_0$, a marking $\mathbf{i}'$ is said to be reachable if there exists a finite sequence of transitions which when fired in succession starting from $\mathbf{i}_0$ can lead to $\mathbf{i}'$. The set of all possible reachable markings of a Petri net, denoted by $\mathcal{S} \subseteq \mathbb{N}^{|\mathcal{P}|}$, is the least fixed point such that $\mathcal{S} = \{\mathbf{i}_0\} \cup \bigcup_{t \in \mathcal{T}} \mathcal{N}_t(\mathcal{S})$.

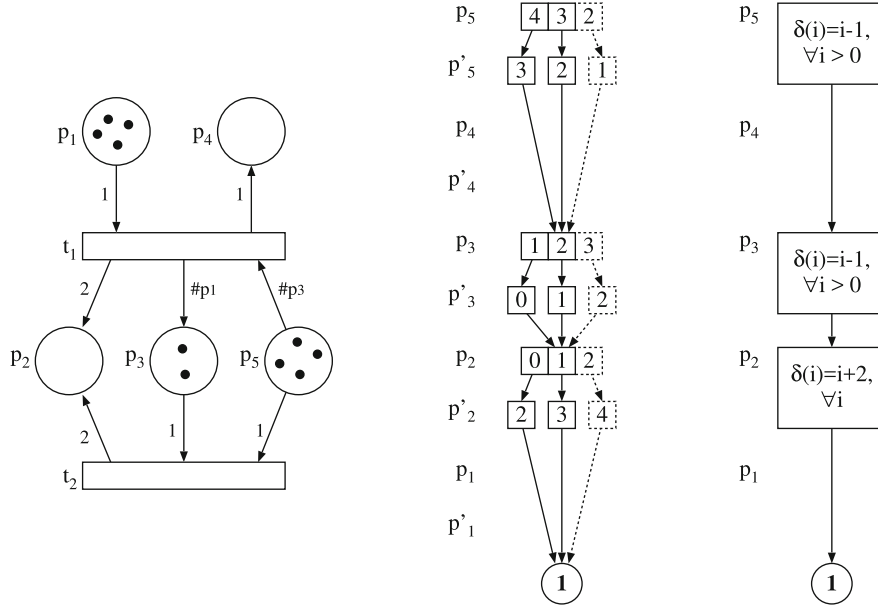


Fig. 1. A Petri net model with marking-dependent arcs, with 2L-level MDD and implicit relation representation of transition t_2 .

The objective of this work is to generate the set $\mathcal{S}$ using saturation-based symbolic state-space generation. The algorithms in the paper are based on the assumption that $\mathcal{S}$ is finite. Therefore, all state variables of the model are bounded, but these bounds do not need to be known a priori as they are discovered during execution of the algorithm.

2.1 Symbolic Representation of State Space and Transition Relations

The state-space of a model is a set of all reachable states, represented symbolically as an indicator function encoded using decision diagrams [19].

Definition 2 (Multi-valued Decision Diagram [13]). Defined over an ordered sequence of L domain variables $(v_L, \dots, v_1)$ such that $v_{k+1} \succ v_k$ and the variable-specific domains are denoted as $\mathcal{D}_k = \{0, 1, \dots, n_k\}$, an ordered MDD is a directed acyclic graph where:

- *Terminal* nodes are labeled **0** or **1** and are associated with a special variable v_0 such that any domain variable $v_k \succ v_0$.
- Every *non-terminal* node $\mathbf{p}$ is associated with some domain variable $\mathbf{p}.var = v_k \succ v_0$ and has $\mathcal{D}_k$ child pointers denoted by $\mathbf{p}[i_k]$ such that $\mathbf{p}.var \succ \mathbf{p}[i_k].var$.

Two nodes, $\mathbf{p}$ and $\mathbf{q}$, in an MDD are said to be *duplicates* if $\mathbf{p}.var = \mathbf{q}.var = v_k$ and $\forall i_k \in \mathcal{D}_k, \mathbf{p}[i_k] = \mathbf{q}[i_k]$. If all pointers of a node point to the same child, then the parent node is termed as *redundant node*. To correctly interpret the unknown but finite variable bounds which are explored during generation, *on-the-fly* saturation requires the use of *quasi-reduced* MDDs, which forbid duplicate nodes but require redundant nodes except for all **0** children: for every non-terminal node $\mathbf{p}$ and for every child i_k , either $\mathbf{p}[i_k] = \mathbf{0}$ or $\mathbf{p}[i_k].var = \mathbf{p}.var - 1$.

Similar to MDDs, a transition relation of a model can be encoded in the form of a set of pairs of states and their next-states using 2L-level MDDs where an indicator function is defined over $(\mathcal{D}_k \times \mathcal{D}_k) \times \dots \times (\mathcal{D}_1 \times \mathcal{D}_1) \rightarrow \{\mathbf{0}, \mathbf{1}\}$. The first set in each pair corresponds to unprimed or “from” state of variable and the second set refers to primed or “to” state of variable.

Given a Petri net encoding $(\mathcal{P}, \mathcal{T}, \mathcal{F}^-, \mathcal{F}^+, \mathcal{F}^o, \mathbf{i}_0)$ of a model, its reachability set, $\mathcal{S}$ and transition relation, $\mathcal{N}$ can be encoded using MDDs:

- The domain variables $(v_L, \dots, v_1)$ of the MDD correspond to the Petri net $\mathcal{P} = (p_1, p_2, \dots, p_{|\mathcal{P}|})$ such that each variable v_k is associated with one p_i , for simplicity.
- The next-state function, $\mathcal{N}_t(\mathbf{i}) = \mathbf{j}$ can be represented by a 2L-level MDD where nodes corresponding to unprimed level and primed level of variable v_k encode the pair (i_k, j_k) .

Nodes in a 2L-level MDD can be redundant or duplicate under same definitions as above. While a 2L-level MDD can be *reduced* as per any choice of reduction rule, the proposed work requires the use of *Fully-Fully* [7] reduced 2L-level MDD which results in skipping of redundant nodes.

Figure 1 shows a small Petri net (left), and symbolic encodings of the relation for transition t_2 using both a 2L-level MDD (middle) and an implicit relation (right). It is noticed that for a Petri net transition where the modified value of a state variable is a function of the variable itself, the 2L-level MDD of the transition assumes a peculiar shape, wherein every edge from the nodes

associated with some primed variable v'_k points to the same child node. This structure creates an unnecessary overhead for on-the-fly saturation as it requires to be rebuilt each time the variable bound expands. In order to encode $\mathcal{N}_t$ of such PNs efficiently, [3] proposes an alternate representation, *implicit relation*. The full encoding of $\mathcal{N}$ contains $|T|$ implicit relations uniquely identified by their topmost nodes, forming an *implicit relation forest*.

Definition 3 (Implicit Relation). An implicit relation defined over the sequence of L domain variables $(v_L, \dots, v_1)$ is a compact version of a 2L-level MDD where the unprimed and primed levels of a variable are combined into a single node, *relation node* which encodes the local effect of the transition on that variable. Each relation node r has a single downward pointer $r.ptr$ to another node such that $r.var \succ r.ptr.var$.

Previous works of Couvreur et al. [9] have proposed *homomorphisms* as an alternative approach to symbolically encode transition relations that works well with Data Decision Diagrams (DDD) [9] and Hierarchical Set Decision Diagrams (SDD) [11] for reachability generation. Homomorphisms are a class of operators which are used on DDDs to encode the function represented by the transition relation. Additionally, this representation is generic and is designed well [10] for extended Petri nets like 2L-level MDDs. Molnar et al. in [14] proposed *abstract next-state diagrams* that encode the transition relations via the use of level-wise *descriptors* to capture the local effect of the transitions. While the domain of models and the key idea of abstracting Petri net transitions is similar to that of implicit relations, the facility with implicit relations to skip the encoding for non-support variables of transitions provides a computational advantage during saturation. The various symbolic representations of transition relations discussed in this section are used as benchmark techniques for performance comparison of the proposed work.

2.2 Saturation

Saturation is a symbolic algorithm for state-space generation that uses DAG-type data structures like decision diagrams or implicit relations for storing and computing operations on the data. In order to calculate the state-space of any system, a breadth first iteration on the fixed point equation $\mathcal{S} = \{\mathbf{i}_0\} \cup \mathcal{N}(\mathcal{S})$, where $\mathcal{N}(\mathcal{S}) = \bigcup_{t \in T} \mathcal{N}_t(\mathcal{S})$ is an explicit method and is computationally inefficient.

However, the strategy of *node saturation* computes the fixed point recursively by exploring reachable states through continuous computation of *relational product* between $\mathcal{S}$ and $\mathcal{N}$ in a bottom-up fashion. The algorithm is described in procedure **Saturate** of Fig. 4. On-the-fly saturation [5, 15, 16] is a variation that explores the variable bounds during reachability set generation and eliminates the need for knowing the variable bounds a priori. However, this dynamic expansion of variable bounds involves certain overhead due to re-construction of $\mathcal{N}$. *Extensible decision diagrams* [20] were introduced to alleviate some of these overheads.

3 Hybrid Relations

Taking into consideration the generality of real-world discrete-state systems and the need to encode these models for efficient execution of on-the-fly saturation techniques, this section formulates an ensemble of decision diagrams and implicit relations for representation of next-state functions where the existence of dependency among variables in the functions and the choice of partitioning technique for the transition relations determine the data structure to be used. This representation combines the power of generalization from 2L-level MDDs and the efficiency of static description from implicit relations. We also modify the on-the-fly saturation algorithm to work with the hybrid relations.

Definition 4 (Hybrid relation). Given a transition $t \in \mathcal{T}$ of a Petri net with a conjunctively-partitioned next-state function $\mathcal{N}_t = \bigcap_{i=1}^{m_t} \mathcal{N}_{t,c_i}$, a hybrid relation encoding of t has a corresponding set of m_t DAGs as follows.

- Each DAG encodes one of the sub-relations $\mathcal{N}_{t,c}$, using either an implicit relation or a 2L-level MDD. Each DAG is uniquely identified by its top-most node.
- Every 2L-level MDD obeys FF-reduction rule for skipping nodes associated with variables $v \notin \text{supp}(\mathcal{N}_{t,c})$.
- Every implicit relation contains a single non-terminal relation node with pointer to the terminal **1**.
- The interpretation of skipped levels is based on the overall hybrid relation of the transition: if a skipped variable v affects the transition at all, then $v \in \text{supp}(\mathcal{N}_t)$, and thus $v \in \text{supp}(\mathcal{N}_{t,c})$ for some c . In this case, the FF-reduction ensures that the intersection is handled appropriately. If instead v does not affect the transition and $v \notin \text{supp}(\mathcal{N}_t)$, then $v \notin \text{supp}(\mathcal{N}_{t,c})$ for any c and the interpretation is “ v is not changed by the firing of t ”.

3.1 Constructing Next-State Functions

We construct the DAG for each sub-relation $\mathcal{N}_{t,c}$ based on the number of variables in its support. The case $\text{supp}(\mathcal{N}_{t,c}) = \emptyset$ corresponds to state variables that do not affect, and are not affected by, the firing of t in the sub-relation; we assume that this case is already handled (because each such variable v will either not affect t at all because $v \notin \text{supp}(\mathcal{N}_t)$, or will affect t but then $v \in \text{supp}(\mathcal{N}_{t,c'})$ for some other c'). When $\text{supp}(\mathcal{N}_{t,c}) = \{v\}$, $\mathcal{N}_{t,c}$ can be represented using an implicit relation node r where $r.\text{var} = v$, the partial function $r.\delta$ encodes the sub-relation and $r.\text{ptr} = \mathbf{1}$. In practice, the implicit relation nodes can be compacted (conjoined) to form a single implicit relation, which will be the original implicit nodes chained together. This has a performance advantage since it decreases the number of sub-relations that needs to be kept track of during saturation for their *symbolic conjunction*. When $|\text{supp}(\mathcal{N}_{t,c})| > 1$, $\mathcal{N}_{t,c}$ must be encoded using a 2L-level MDD over the domain variables in $\text{supp}(\mathcal{N}_{t,c})$.

Different methods may be used for partitioning a transition relation $\mathcal{N}_t$ into its sub-relations. One method [16] is to use the finest possible partition, such that

each state variable belongs to the support of at most one sub-relation. Another method [7] separates the transition enabling and firing conditions, and then uses the finest possible partition so that each variable is *modified* in at most one sub-relation (but may appear, read-only, in other sub-relations). These methods are illustrated in Fig. 2 for transition t_1 from the PN in Fig. 1. The left side uses the method from [16], and obtains two sub-relations: one as a chain of implicit nodes, representing $p'_4 = p_4 + 1$ and $p'_2 = p_2 + 2$, and one as an MDD, representing $(p_5 \geq p_3) \wedge (p'_5 = p_5 - p_3) \wedge (p'_3 = p_3 + p_1) \wedge (p_1 \geq 1) \wedge (p'_1 = p_1 - 1)$. The right side uses the method from [7], and obtains three sub-relations: one representing $p'_3 = p_3 + p_1$, one representing $(p_5 \geq p_3) \wedge (p'_5 = p_5 - p_3)$, and a chain of implicit nodes representing $p'_4 = p_4 + 1$, $p'_2 = p_2 + 2$, and $(p_1 \geq 1) \wedge (p'_1 = p_1 - 1)$.

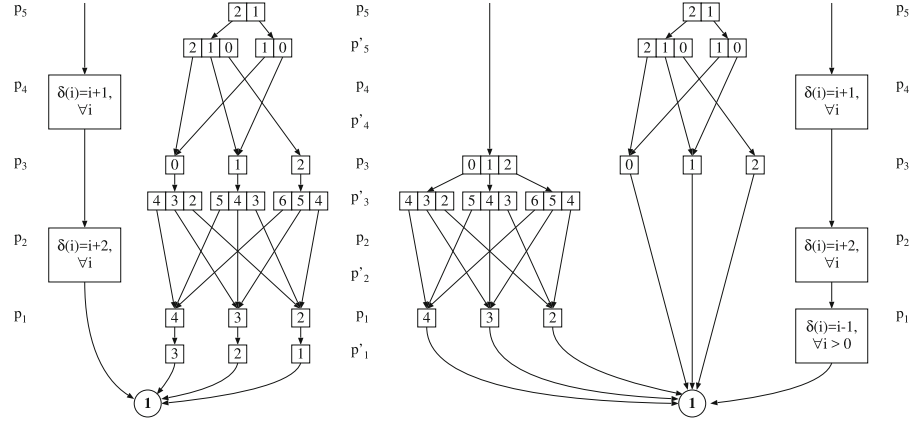


Fig. 2. Hybrid relation-based encodings of transition t_1 from Fig. 1 when partitioning techniques from [16] (left) and [7] (right) are used.

Algorithm **BuildHybrid**, shown in Fig. 3, constructs an overall hybrid relation for a given transition $t \in \mathcal{T}$, as a set of DAGs. This should be called once, for each transition, before saturation is invoked. The set of sub-relation DAGs for transition t is denoted by $\mathcal{N}_t.nodes$. Note that the overall transition relation $\mathcal{N}_t$ is not computed, but rather is evaluated during the saturation algorithm on the fly. The hybrid relations are updated during saturation as new local states are discovered, using algorithm **Confirm**, also shown in Fig. 3.

3.2 Saturation Using Hybrid Relations

Saturating a node for variable v_k requires repeatedly firing transitions in $\mathcal{T}_k$, the set of transitions whose top-most variable is v_k (i.e., $supp(\mathcal{N}_t) \cap \{v_L, \dots, v_k\} = \{v_k\}$), until a fixed point is reached. The algorithm is shown in Fig. 4. Procedures **RecFire** and **Saturate** are similar to those of saturation using MxDs [16] (using either a single DAG per transition, or a single DAG for all of $\mathcal{T}_k$), except all nodes

```

BuildHybrid(transition t)
• Builds hybrid relation for Petri net transition t

1: Partition  $\mathcal{N}_t$  into  $\mathcal{N}_{t,c_1}, \mathcal{N}_{t,c_2}, \dots, \mathcal{N}_{t,c_{m_t}}$ 
2:  $n \leftarrow \mathbf{1}$ ;
3: for each  $\mathcal{N}_{t,c}$  s.t.  $|supp(\mathcal{N}_{t,c})| = 1$ , from bottom up, do
4:    $r \leftarrow$  new relation node;
5:    $r.var \leftarrow \mathcal{N}_{t,c}.var$  ;
6:    $r.\delta \leftarrow \mathcal{N}_{t,c}.\Delta$  ;
7:    $r.ptr \leftarrow n$ ;
8:    $n \leftarrow r.ptr$ ;
9:  $\mathcal{N}_t.nodes \leftarrow \{n\} \setminus \{\mathbf{1}\}$ ;
10: for each  $\mathcal{N}_{t,c}$  s.t.  $|supp(\mathcal{N}_{t,c})| > 1$  do
11:    $s_{from} \leftarrow s_0$ ;
12:    $s_{to} \leftarrow \mathcal{N}_{t,c}(s_{from})$ ;
13:    $\mathcal{N}_{t,c}.updateHybrid(s_{from}, s_{to})$ ;
14:    $\mathcal{N}_t.nodes \leftarrow \mathcal{N}_t.nodes \cup \mathcal{N}_{t,c}.node$ ;

Confirm(level k, index i)
• Updates relevant relations  $\mathcal{N}_t, \mathcal{N}_{t,c}$  if local state  $v_k = i$  is new

1: if  $i \notin \mathcal{D}_k$  then
2:    $\mathcal{D}_k \leftarrow \mathcal{D}_k \cup \{i\}$ ;
3:   for each  $t \in \mathcal{T}$  do
4:     for each  $\mathcal{N}_{t,c}$  s.t.  $|supp(\mathcal{N}_{t,c})| > 1$  and  $k \in supp(\mathcal{N}_{t,c})$  do
5:        $\mathcal{N}_t.nodes \leftarrow \mathcal{N}_t.nodes \setminus \mathcal{N}_{t,c}.node$ ;
6:       for each  $s_{from} \in \mathcal{N}_{t,c}.buildState(k, i)$ ;
7:          $s_{to} \leftarrow \mathcal{N}_{t,c}(s_{from})$  ;
8:          $\mathcal{N}_{t,c}.updateHybrid(s_{from}, s_{to})$ 
9:        $\mathcal{N}_t.nodes \leftarrow \mathcal{N}_t.nodes \cup \{\mathcal{N}_{t,c}.node\}$ ;

```

Fig. 3. Algorithms for building a hybrid relation.

of the hybrid relation for a transition are traversed simultaneously. Thus, in addition to the MDD node encoding the set of states, procedure **RecFire** requires a set of hybrid relation nodes, rather than a single MxD node (as used in [16]) or a transition identifier (as used in [5]). This also means that the compute table entries for **RecFire** must include the set of hybrid relation nodes (c.f. lines 2 and 15).

The actual firing with a set of hybrid relation nodes is handled in helper procedure **Fire**, also shown in Fig. 4. This takes in a set of hybrid relation nodes, and determines the set of nodes to follow one level below, by examining each node in the set and following its downward pointer ($h[i][j]$ in line 4), unless the current level is skipped, in which case the current node is used again (line 3). Note that $h[i][j]$ must be determined differently, based on if h is an implicit node, or an MDD node. If the set of downward pointers contains terminal node $\mathbf{0}$, then the intersection of the sub-relations will be empty, and in this case no states will be reached (line 5). Otherwise, the traversal continues by calling **RecFire** recursively

<pre> mdd Fire(mdd n, nodeset $\mathcal{H}$, index i, j) • Fire hybrid relation nodes $\mathcal{H}$ on node n, from i to j. 1: $\mathcal{H}' \leftarrow \emptyset$; 2: for each $h \in \mathcal{H}$ do 3: if $n.var > h.var$ then $\mathcal{H}' \leftarrow \mathcal{H}' \cup \{h\}$; 4: else $\mathcal{H}' \leftarrow \mathcal{H}' \cup \{h[i][j]\}$; 5: if $\mathcal{H}' = \emptyset$ or $\mathbf{0} \in \mathcal{H}'$ then return $\mathbf{0}$; 6: $f \leftarrow \text{RecFire}(n[i], \mathcal{H}')$; 7: if $f \neq \mathbf{0}$ then Confirm($n.var, j$); 8: return f; </pre>	
<pre> Saturate(level k, mdd n) • Saturate node n at level k using $\mathcal{T}_k$. 1: $\mathcal{Q} \leftarrow \{i : n[i] \neq \mathbf{0}\}$; 2: while $\mathcal{Q} \neq \emptyset$ do 3: $i \leftarrow \text{Choose}(\mathcal{Q})$; 4: $\mathcal{Q} \leftarrow \mathcal{Q} \setminus \{i\}$; 5: for each $t \in \mathcal{T}_k$ do 6: $h^t \leftarrow \text{topmost}(\mathcal{N}_t.nodes)$; 7: for each j s.t. $h^t[i][j] \neq \mathbf{0}$ do 8: $f \leftarrow \text{Fire}(n, \mathcal{N}_t.nodes, i, j)$; 9: $u \leftarrow \text{Union}(f, n[j])$; 10: if $u \neq n[j]$ then 11: $n[j] \leftarrow u$; 12: $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{j\}$; </pre>	<pre> mdd RecFire(mdd n, nodeset $\mathcal{H}$) • Fire $\mathcal{H}$ on node n and then saturate 1: if $n = \mathbf{0}$ or $\mathcal{H} = \{\mathbf{1}\}$ then return n; 2: if $(\text{RecF}, n, \mathcal{H}, m) \in \text{CT}$ then return m; 3: $h^t \leftarrow \text{topmost}(\mathcal{H})$; 4: $k \leftarrow \max(n.var, h^t.var)$; 5: $m \leftarrow \text{new MDD node for variable } v_k$; 6: if $n.var > h^t.var$ then 7: for each $i \in \mathcal{D}_k$ do 8: $m[i] \leftarrow \text{RecFire}(n[i], \mathcal{H})$; 9: else 10: for each i, j s.t. $n[i] \neq \mathbf{0}, h^t[i][j] \neq \mathbf{0}$ 11: $f \leftarrow \text{Fire}(n, \mathcal{H}, i, j)$; 12: $m[j] \leftarrow \text{Union}(m[j], f)$; 13: Saturate($k, m$); 14: $m \leftarrow \text{UniquelyInsert}(m)$; 15: $\text{CT} \leftarrow \text{CT} \cup (\text{RecF}, n, \mathcal{H}, m)$; 16: return m; </pre>

Fig. 4. Saturation using hybrid relations

(line 6). If the reached set of states is non-empty, then local state variable j must be confirmed [5] if it is new (line 7).

4 Experimental Results

4.1 Setup

We implemented the proposed hybrid relation-based saturation algorithm, HybSat, using the [16] partitioning method in SMART [6] using Meddly [2] as the underlying decision diagram library. We conducted two groups of experiments to compare the performance of HybSat,

- **Group I**, with the existing “on-the-fly saturation with matrix diagrams” approach (OtfSat) from SMART and with DDD-based reachability set generation from ITS-Tools (ITSTools) for extended Petri net models on identical static variable order for each run.

- **Group II**, with the saturation algorithm based on a generalized representation of next-state functions (**AbstractDesc**) from [17] for ordinary Petri net models using SOUPS [18] ordering for variables.

Each Petri net model has multiple instances described by parameters that determine the size of the model or its initial state. Each experimental run comprises of running an instance of a model on the group-specific set of tools. SMART and ITS-Tools are run on their default settings from MCC, except that the use of SDD in ITS-Tools is disabled since it requires additional structural information about the model. The tool for **AbstractDesc** is Java-based and is allocated a maximum heap size of 6 GB and stack size of 512 MB for its experiments. All experiments are run on a server of Intel Xeon CPU 2.13 GHz with 48G RAM under Linux Kernel 4.9.9 with timeout for each run set to 1 h.

For Group II experiments, we obtained the ordinary Petri net models from the model repository of [1]. The suite of extended Petri nets used for the experiments in Group I category are described as follows:

- *Swap* model where given a list of N distinct integers, operations are to exchange two neighboring integers. The model contains N places and $N-1$ transitions, where both arcs of each transition have marking-dependent cardinalities.
- *Stack* model that generates the count of possible combination of D objects that can have values in the range from 1 to N . The PN has D places and 4 transitions, where **inc** & **dec** control that the number of tokens are within limits of N and **push** and **pop** move the tokens among the places of the net.
- A simplified version of *Leader* election protocol [12] that designates a single process as the leader among a ring of N processes by determining the maximum of the *unique_ids* transmitted by the processes.
- A non-stochastic version of *FMS* (Flexible Manufacturing System) from [8] that has N number of 3 different kinds of parts fitted on three machines.
- *Tiles* models a $N \times M$ rectangular puzzle with movable tiles. The model has $N \times M$ places and 4 transitions (**left**, **right**, **up**, **down**) to represent position and movement of the tiles.
- *Tower of Hanoi* models D disks of different sizes that can move across three rods with constraints on sizes of the disks that can be placed on a rod.
- A modified version of *SatelliteMemory*, an ordinary Petri net from [1], where the constant arc cardinalities are converted to constant places and markings of these places are used to define the arc cardinalities in order to introduce synthetic marking-dependent behavior.

4.2 Observations

Table 1 and Table 2 summarize the experiments run on extended Petri nets and ordinary Petri nets respectively. The metrics used for performance comparison include the time taken by the reachability generation algorithm and the total number of peak nodes required by the execution for encoding the transition relations. For experiments conducted under Group I (see Table 1), the observations

from **OtfSat** and **HybSat** reveal that models that have events with tightly-bound sub-event effects, force the **HybSat** to build 2L-level MDDs leading to similar metric values and often these values are worse with respect to **ITSTools**. For example, models like **Stack**, the sub-events display a chain-dependency among the variables leading to creation of a full 2L-level MDD for encoding the relation. For models such as **FMS**, where all but four transitions have constant arc cardinalities, **HybSat** performs better than **ITSTools** but shows similar performance with **OtfSat**. The overall results in these experiments indicate that higher the number of relation nodes in the encoding, better is the performance of **HybSat**. However, the ability of **HybSat** to encode general transitions make it as versatile as **OtfSat** and **ITSTools**.

The observations of Group II experiments that are summarized in Table 2, confirm that within a given model, the ratio of execution times of saturation algorithm in the Java implementation of **AbstractDesc** to that of **HybSat** in **SMART** is fairly constant. While these encodings and algorithms are conceptually similar, the significant yet constant difference in the execution times can be attributed to the implementational details and underlying language. Note that the number of nodes in case of **AbstractDesc** is higher compared to **HybSat** because the former does not skip descriptors that encode identity relations and it counts every pair of “from” and “to” local states as defined by the descriptors. The number of nodes encoded for $\mathcal{N}$ by **HybSat** for is at most equal to the number of arcs in the Petri net definition. Since **HybSat** pre-determines the encoding strategy for each transition, for all models under this experiment group, their transitions are represented as relation nodes. Hence, the comparison metrics for reachability generation in **HybSat** and **ImplSat** are exactly the same and therefore, the observations of **ImplSat** are omitted. The missing fields refer to timeout of the experimental run.

5 Conclusions

In this paper, we introduced hybrid relations, an ensemble data-structure, for symbolic encoding of transition relations in a general class of discrete-state systems. The translation of a system event into its corresponding hybrid relation, involves structural analysis of the event in terms of dependencies among the variables of the event. This prior system review to assign a suitable data-structure for encoding each (sub)event rewards during saturation, since unnecessary overhead costs of managing the compute-table entries for 2L-level MDD operations and saturation are eliminated. The efficiency and validity of the proposed saturation algorithm is empirically evaluated with respect to several versions of saturation algorithm.

Table 1. Performance comparison of reachability generation among SMART (OtfSat, HybSat) and ITSTools from ITS-tools over benchmark of extended Petri net models

Instance	S	Time (in sec)			Peak Nodes: $\mathcal{N}$		
		OtfSat	ITSTools	HybSat	OtfSat	ITSTools	HybSat
Swap							
10	3.6×10^6	2.7×10^{-01}	9×10^{-01}	3.3×10^{-01}	1149	58	1091
12	4.8×10^8	3.0×10^{00}	9.5×10^{00}	3.5×10^{00}	1890	70	1861
15	1.3×10^{12}	3.2×10^{02}	4.0×10^{02}	2.9×10^{02}	3360	88	3586
Stack							
D8 V5	3.9×10^5	2.7×10^{00}	6.0×10^{-02}	2.7×10^{00}	290	41	242
D10 V3	5.2×10^{47}	6.3×10^2	5.0×10^{00}	5.7×10^2	2763	409	2120
Leader							
7	2.39×10^7	4.0×10^{01}	5.8×10^{01}	6.1×10^{01}	1109	79	476
8	3.0×10^8	1.9×10^{02}	2.3×10^{02}	2.7×10^{02}	1676	105	735
Flexible manufacturing system							
15	2.7×10^{11}	2.9×10^{-01}	5.3×10^{-01}	1.7×10^{-01}	1544	118	355
20	6.0×10^{12}	4.2×10^{-01}	1.29×10^{00}	3.9×10^{-01}	1660	118	505
30	7.7×10^{14}	9.1×10^{-01}	5.51×10^{00}	1.1×10^{00}	2800	118	880
40	2.6×10^{16}	6.4×10^{00}	1.3×10^{01}	5.3×10^{00}	7342	118	1355
Tiles							
N3M3	1.8×10^{05}	1.4×10^{00}	8.9×10^{00}	2.2×10^{00}	952	154	362
N5M2	1.8×10^{06}	1.5×10^{01}	5.5×10^{01}	1.3×10^{01}	1223	168	433
N3M4	2.4×10^{08}	2.3×10^{01}	3.1×10^{01}	1.7×10^{01}	2561	212	641
Tower of Hanoi							
D8	6.5×10^{03}	3.7×10^{00}	5.9×10^{00}	4.3×10^{00}	4566	40	1219
D10	5.9×10^{04}	5.4×10^{01}	7.0×10^{01}	5.9×10^{01}	9837	40	2481
Satellite							
X 100 Y 3	7.6×10^{04}	2.1×10^{01}	1.1×10^{01}	1.9×10^{01}	3207	98	737

Although this paper briefly discusses the use of event partitioning techniques for building hybrid relations, the consequence of each technique on the efficiency of hybrid relations based saturation algorithm is yet to be explored. This can pave way for applying desirable partitioning methods for each event instead of making an apriori choice for all events. Building and incorporating compatible data structures into hybrid relations that can optimally utilize certain event behaviors can make the representational formalism more powerful and efficient for saturation-based symbolic reachability generation.

Table 2. Performance comparison of reachability generation among SMART (ImpSat, HybSat) and AbstractDesc over benchmark of ordinary Petri net models

Model	S	Time (in sec)	Peak Nodes for encoding $\mathcal{N}$		
		AbstractDesc	HybSat	AbstractDesc	HybSat
Eratosthenes					
200	1.1×10^{46}	4.09×10^{-01}	2.27×10^{-02}	441	603
500	4.1×10^{121}	2.32×10^{00}	2.35×10^{-01}	2820	1875
Philosophers					
1000	1.3×10^{477}	2.24×10^{01}	3.98×10^{00}	62410	15000
2000	4.0×10^{2385}	7.03×10^{01}	2.41×10^{01}	124917	30000
SwimmingPool					
10	3.4×10^{10}	5.87×10^{01}	3.97×10^{01}	721877	20
Kanban					
200	3.2×10^{22}	1.26×10^{01}	1.92×10^{00}	44028	39
500	7.1×10^{26}	1.51×10^{02}	2.94×10^{01}	260027	39
HouseConstruction					
20	1.4×10^{13}	1.27×10^{01}	3.06×10^{00}	279043	51
50	1.6×10^{19}	1.74×10^{02}	4.37×10^{01}	2280222	51
SmallOperatingSystem					
MT512 DC128	1.0×10^{11}	1.51×10^{02}	4.05×10^{01}	348039	22
MT512 DC256	2.5×10^{11}	3.13×10^{02}	1.03×10^{02}	594183	22
GPPP					
C10 N10	2.4×10^{10}	7.1×10^{00}	9.99×10^{00}	173153	79
C10 N100	1.8×10^{11}	7.25×10^{00}	1.34×10^{01}	194454	79
Referendum					
100	5.2×10^{47}	4.48×10^{-01}	1.57×10^{-02}	2701	460
200	2.7×10^{95}	3.14×10^{01}	9.07×10^{00}	529255	940
TCPcondis					
15	5.4×10^{12}	3.29×10^{02}	8.48×10^{01}	4519532	90
20	5.6×10^{14}	–	3.03×10^{02}	–	90
Raft					
6	2.9×10^{26}	–	1.87×10^{01}	–	620
7	3.6×10^{35}	–	1.46×10^{02}	–	829
Peterson					
2	2.1×10^{04}	1.79×10^{00}	1.50×10^{01}	18055	298
3	3.4×10^{06}	1.29×10^{02}	2.85×10^{01}	1451193	739
Airplane					
200	2.8×10^{08}	1.21×10^{02}	1.08×10^{01}	1208062	2172
500	4.3×10^{09}	–	4.44×10^{00}	–	6376
Dekker					
15	2.8×10^{05}	7.05×10^{02}	3.99×10^{01}	7084909	729
20	1.2×10^{07}	5.21×10^{02}	1.31×10^{01}	3565360	1365

Acknowledgment. This work was supported in part by the National Science Foundation under grant ACI-1642327.

References

1. MCC: Model Checking Competition @ Petri Nets. <https://mcc.lip6.fr>
2. Babar, J., Miner, A.S.: MEDDLY: multi-terminal and edge-valued decision diagram LibrarY. In: Proceedings of the QEST, pp. 195–196. IEEE Computer Society (2010)
3. Biswal, S., Miner, A.S.: Improving saturation efficiency with implicit relations. In: Donatelli, S., Haar, S. (eds.) PETRI NETS 2019. LNCS, vol. 11522, pp. 301–320. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-21571-2_17
4. Ciardo, G., Lüttgen, G., Siminiceanu, R.: Saturation: an efficient iteration strategy for symbolic state—space generation. In: Margaria, T., Yi, W. (eds.) TACAS 2001. LNCS, vol. 2031, pp. 328–342. Springer, Heidelberg (2001). https://doi.org/10.1007/3-540-45319-9_23
5. Ciardo, G., Marmorstein, R., Siminiceanu, R.: Saturation unbound. In: Garavel, H., Hatchiff, J. (eds.) TACAS 2003. LNCS, vol. 2619, pp. 379–393. Springer, Heidelberg (2003). https://doi.org/10.1007/3-540-36577-X_27
6. Ciardo, G., Miner, A.S.: SMART: Stochastic Model checking Analyzer for Reliability and Timing, User Manual. <http://smart.cs.iastate.edu>
7. Ciardo, G., Yu, A.J.: Saturation-based symbolic reachability analysis using conjunctive and disjunctive partitioning. In: Borrione, D., Paul, W. (eds.) CHARME 2005. LNCS, vol. 3725, pp. 146–161. Springer, Heidelberg (2005). https://doi.org/10.1007/11560548_13
8. Cimatti, A., Clarke, E., Giunchiglia, F., Roveri, M.: NuSMV: a new symbolic model verifier. In: Halbwachs, N., Peled, D. (eds.) CAV 1999. LNCS, vol. 1633, pp. 495–499. Springer, Heidelberg (1999). https://doi.org/10.1007/3-540-48683-6_44
9. Couvreur, J.-M., Encrenaz, E., Paviot-Adet, E., Poitrenaud, D., Wacrenier, P.-A.: Data decision diagrams for Petri Net analysis. In: Esparza, J., Lakos, C. (eds.) ICATPN 2002. LNCS, vol. 2360, pp. 101–120. Springer, Heidelberg (2002). https://doi.org/10.1007/3-540-48068-4_8
10. Couvreur, J.-M., Thierry-Mieg, Y.: Hierarchical decision diagrams to exploit model structure. In: Wang, F. (ed.) FORTE 2005. LNCS, vol. 3731, pp. 443–457. Springer, Heidelberg (2005). https://doi.org/10.1007/11562436_32
11. Hamez, A., Thierry-Mieg, Y., Kordon, F.: Hierarchical set decision diagrams and automatic saturation. In: van Hee, K.M., Valk, R. (eds.) PETRI NETS 2008. LNCS, vol. 5062, pp. 211–230. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-68746-7_16
12. Itai, A., Rodeh, M.: Symmetry breaking in distributed networks. In: 22th Annual Symposium on Foundations of Computer Science, pp. 150–158. IEEE Computer Society Press, October 1981
13. Kam, T., Villa, T., Brayton, R.K., Sangiovanni-Vincentelli, A.: Multi-valued decision diagrams: theory and applications. *Mult. Valued Log.* 4(1–2), 9–62 (1998)
14. Marussy, K., Molnár, V., Vörös, A., Majzik, I.: Getting the priorities right: saturation for prioritised Petri Nets. In: van der Aalst, W., Best, E. (eds.) PETRI NETS 2017. LNCS, vol. 10258, pp. 223–242. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-57861-3_14
15. Minato, S.-i.: Zero-suppressed BDDs and their applications. *Softw. Tools Technol. Transf.* 3, 156–170 (2001)

16. Miner, A.S.: Saturation for a general class of models. *IEEE Trans. Softw. Eng.* **32**(8), 559–570 (2006)
17. Molnár, V., Majzik, I.: Saturation enhanced with conditional locality: application to Petri Nets. In: Donatelli, S., Haar, S. (eds.) *PETRI NETS 2019*. LNCS, vol. 11522, pp. 342–361. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-21571-2_19
18. Smith, B., Ciardo, G.: SOUPS: a variable ordering metric for the saturation algorithm. In: *2018 18th International Conference on Application of Concurrency to System Design (ACSD)*, pp. 1–10 (2018)
19. Somenzi, F.: Binary decision diagrams. In: *Computational System Design*. NATO Science Series F: Computer and Systems Sciences, vol. 173, pp. 303–366. IOS Press (1999)
20. Wan, M., Ciardo, G.: Symbolic state-space generation of asynchronous systems using extensible decision diagrams. In: Nielsen, M., Kučera, A., Miltersen, P.B., Palamidessi, C., Tůma, P., Valencia, F. (eds.) *SOFSEM 2009*. LNCS, vol. 5404, pp. 582–594. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-540-95891-8_52